

# Working with ESP-IDF in CLion

AUTHOR

**Oleg Zinovyev**

Technical and content writer at JetBrains



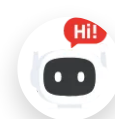
This tutorial shows how to work with an ESP-IDF project in CLion, a cross-platform C and C++ IDE. We'll build an application, flash it to an ESP32-S3 board, and debug it. We'll also run the serial monitor and examine the configuration menu that allows you to customize your ESP-IDF project. All of this will be done within the IDE, without having to switch to a system terminal or any other tool.

The tutorial is beginner-friendly, so it's okay if you've never worked with ESP-IDF and CLion.

## Prerequisites

We'll use the following hardware:

- ESP32-S3-DevKitC-1 v1.1.
- MacBook Pro M2 Max.
- USB-C to micro USB data cable.



To get started:

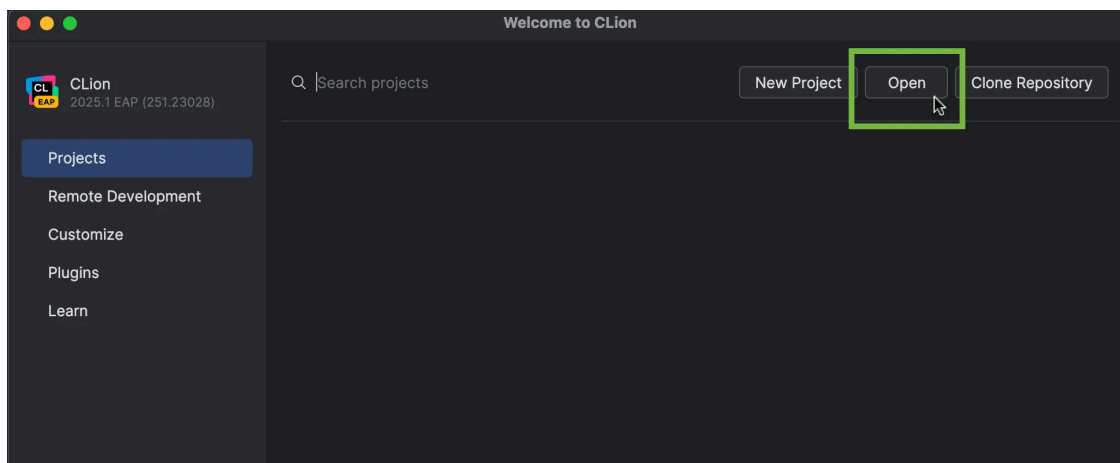


whether there are any **discounted or free options** available.

Although we'll be running CLion on macOS, the workflow and settings are generally similar to those for Windows and Linux, except in cases mentioned in this article. If you want to learn more about the configuration options available in CLion beyond those demonstrated in this tutorial, refer to the **CLion documentation**.

## Configuring an ESP-IDF project

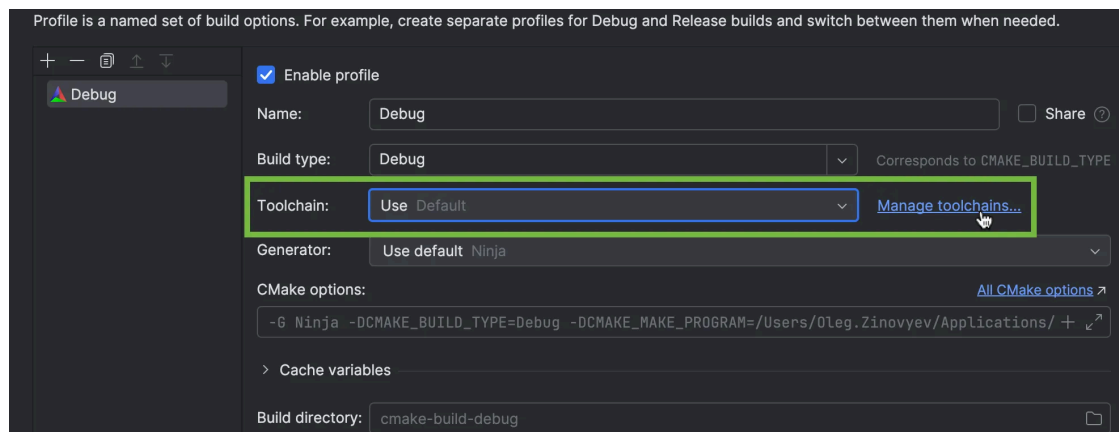
1. Run CLion.
2. Select **open** from the welcome screen.



3. Navigate to the default ESP-IDF directory on your computer. For the purposes of this tutorial, this is `/Users/username/esp/esp-idf`. Then, go to the `examples` subdirectory and select a project to build.

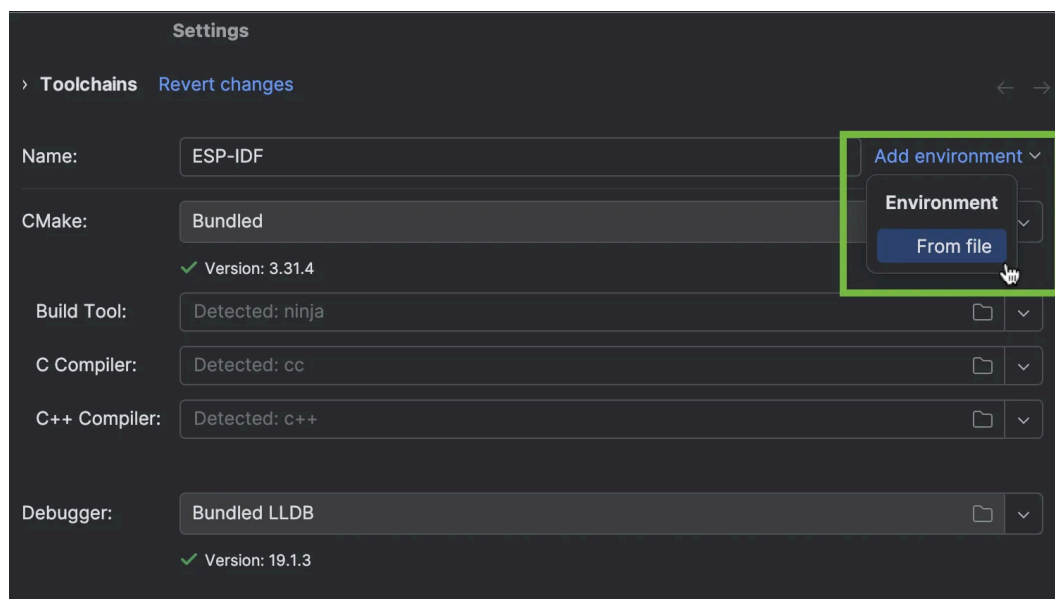
In this tutorial, we'll be using `led_strip_simple_encoder`. It's located in `examples/peripherals/rmt`. This application generates an LED rainbow chase on the board. Although it's intended to be used with an LED strip, it also works with a single LED board like the one used here. The LED will simply flash different colors in a predetermined sequence.

4. Click **Trust Project**. The **Open Project Wizard** will then open.
5. Click **Manage toolchains...**

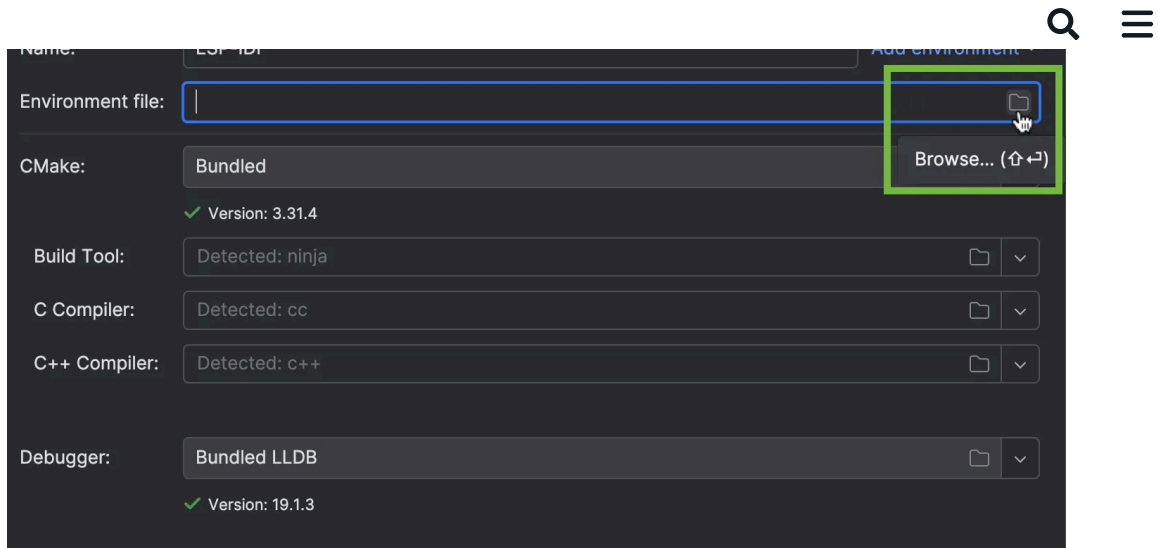


6. Click **+**, and select **System** (for Windows, select **MinGW** instead) to create a new toolchain. You can name it anything you like.

- Select **Add environment** > **From file** in the new toolchain template.



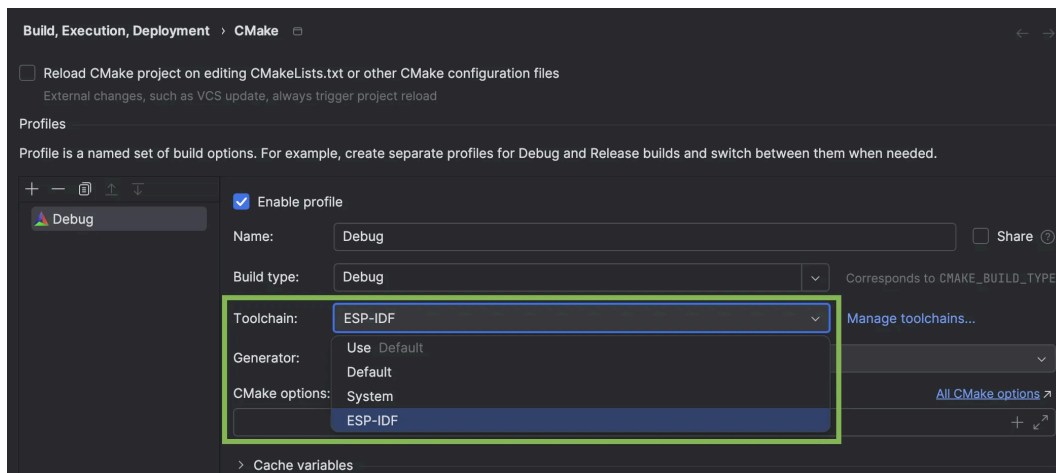
- Click **Browse...**



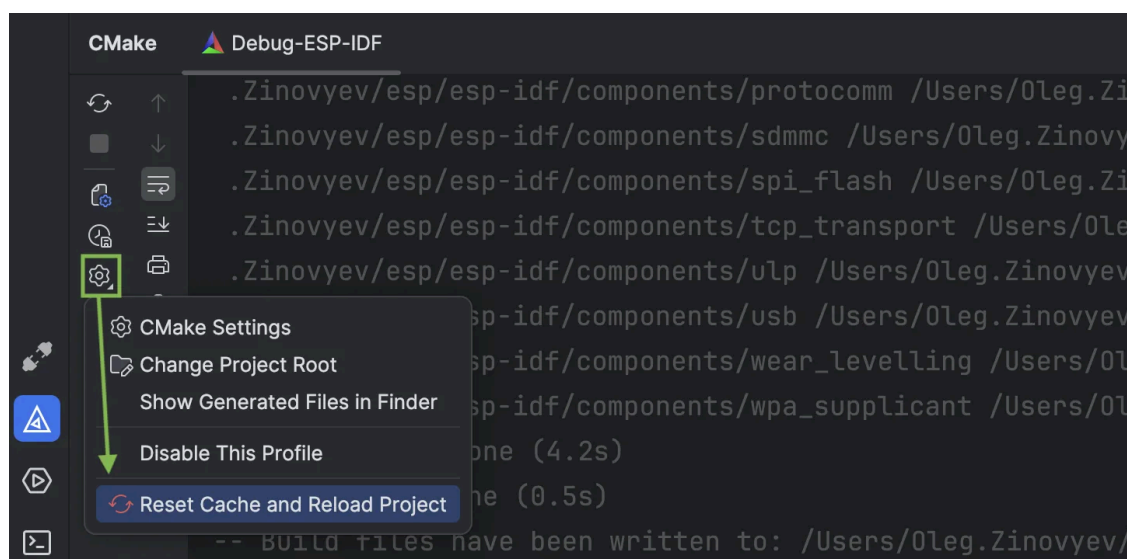
- Select the environment file on your computer. For macOS, you need the file called **export.sh** (on Windows, it's **export.bat**), which is located in the default ESP-IDF directory.
- Click **Apply**

#### 7. Go to **Settings** > **Build, Execution, Deployment** > **CMake**.

- In the default **Debug** profile settings, select the recently created toolchain, which in our case is **ESP-IDF**.



- In the **CMake options** field, enter **-DIDF\_TARGET=esp32s3** (because an ESP32-S3-based board is used).
- In the **Build directory** field, enter **build**.
- Click **OK**.



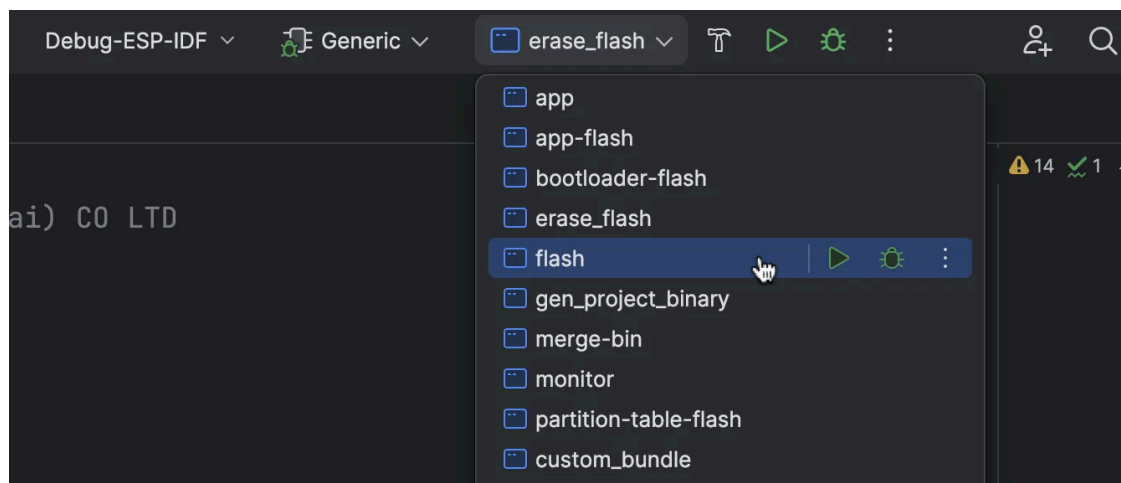
If the project is loaded successfully, you'll see **[Finished]** at the end of the CMake log. Your application is ready to be built and flashed to the board.

## Building the application and flashing the board

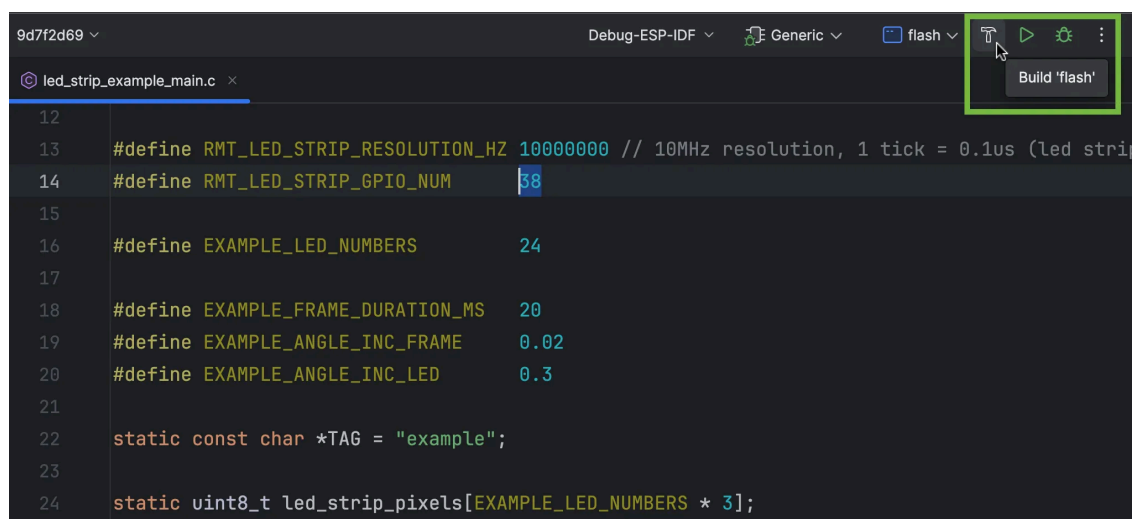
1. Make sure your board is connected to your computer via the UART port.
2. If you're using the same application example, make sure that the GPIO LED number is correctly specified in the source code:
  - In CLion's **Project** tool window, locate the main directory in your project directory and open the `led_strip_example_main.c` file.
  - In the `#define RMT_LED_STRIP_GPIO_NUM` line, change the default value to `38` or `48`, depending on your board hardware revision.

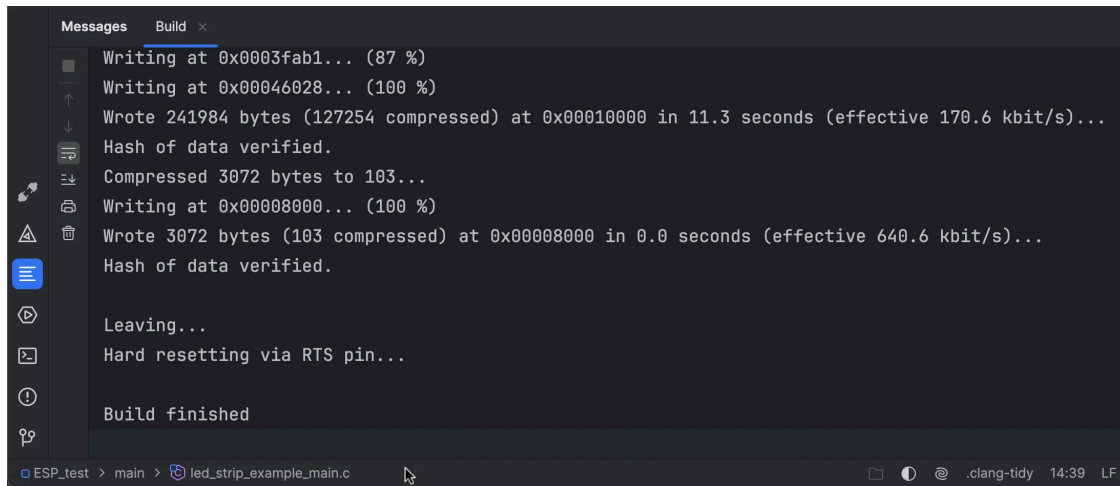
```
4  * SPDX-License-Identifier: Unlicense OR CC0-1.0
5  */
6  > #include ...
12
13  #define RMT_LED_STRIP_RESOLUTION_HZ 10000000 // 10MHz resolution, 1 tick =
14  #define RMT_LED_STRIP_GPIO_NUM 38
15
16  #define EXAMPLE_LED_NUMBERS 24
17
18  #define EXAMPLE_FRAME_DURATION_MS 20
```

3. Click the **Run / Debug Configurations** drop-down list on the main toolbar and select the **flash** configuration. This configuration allows you to build the project and then flash the board automatically.



4. Click the green **Build** icon on the main IDE toolbar.





```
Messages Build x
Writing at 0x0003fab1... (87 %)
Writing at 0x00046028... (100 %)
Wrote 241984 bytes (127254 compressed) at 0x00010000 in 11.3 seconds (effective 170.6 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 103...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (103 compressed) at 0x00008000 in 0.0 seconds (effective 640.6 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...

Build finished

ESP_test > main > led_strip_example_main.c clang-tidy 14:39 LF
```

After the build, the board will blink according to the configured rainbow-chase pattern.

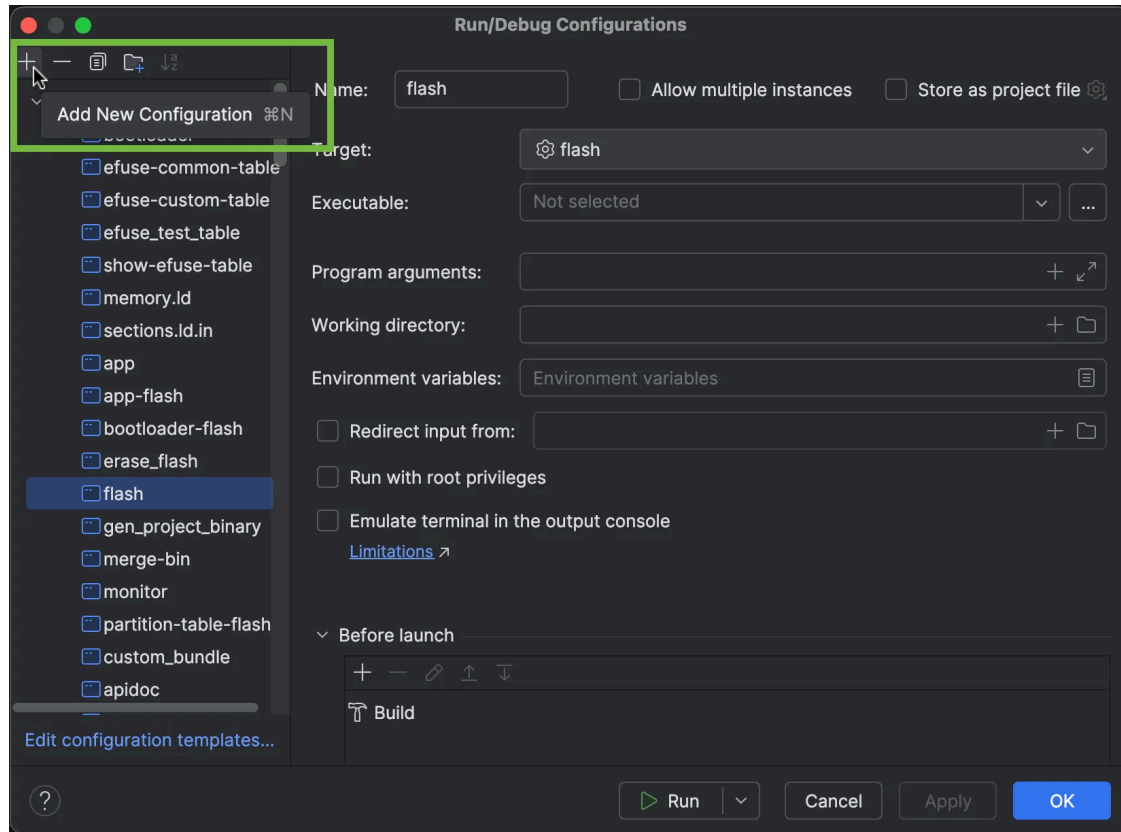


To change the chasing speed, update the `EXAMPLE_ANGLE_INC_FRAME` value in `led_strip_example_main.c`. To change the density of colors, update `EXAMPLE_ANGLE_INC_LED` in the same file.

## Running the IDF monitor

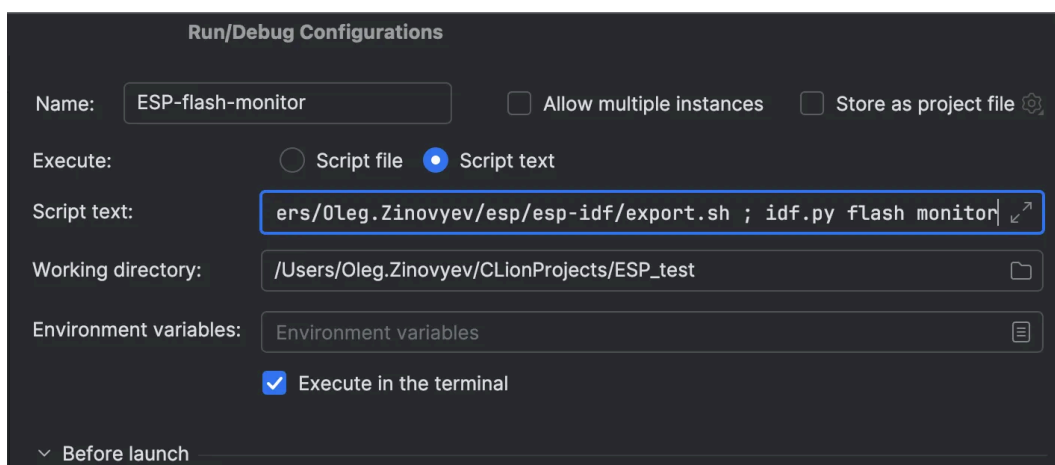


2. GO TO **Run | Edit Configurations** AND CLICK **Add New Configuration**.

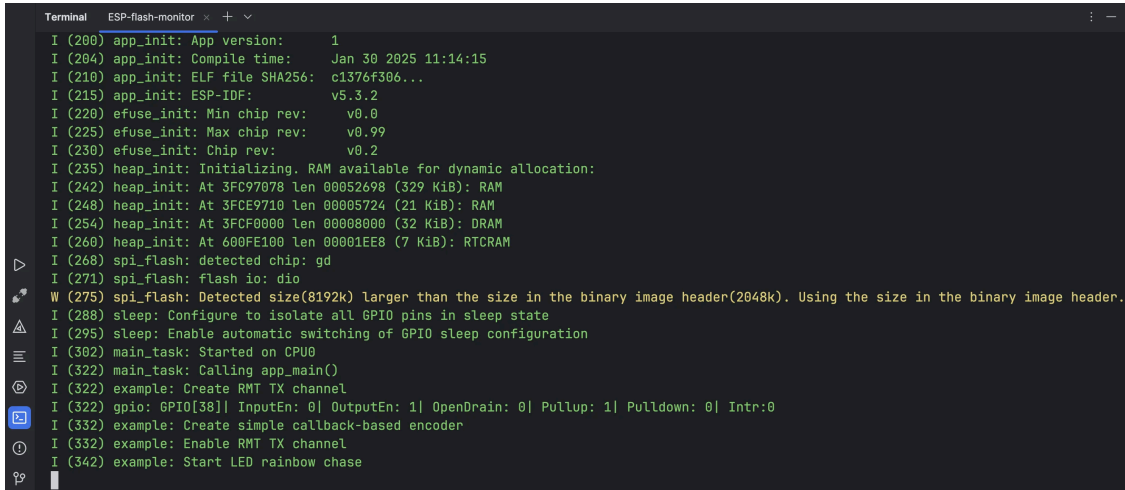


3. Select the **Shell Script** template. In this new configuration dialog:

- Enter a name of your choice.
- Select **Script text** next to **Execute**.
- Enter the following text, including the path of the environment file you've just copied: `./Users/Oleg.Zinovyev/esp/esp-idf/export.sh ; idf.py flash monitor`.



You'll then see the diagnostic data from the monitor displayed in the IDE's terminal.



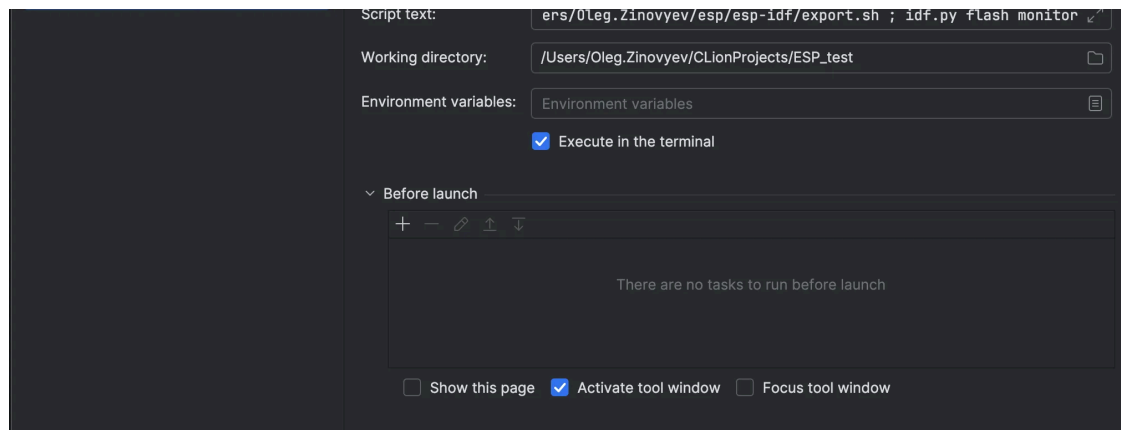
```
Terminal  ESP-flash-monitor x + -
I (200) app_init: App version: 1
I (204) app_init: Compile time: Jan 30 2025 11:14:15
I (210) app_init: ELF file SHA256: c1376f306...
I (215) app_init: ESP-IDF: v5.3.2
I (220) efuse_init: Min chip rev: v0.0
I (225) efuse_init: Max chip rev: v0.99
I (230) efuse_init: Chip rev: v0.2
I (235) heap_init: Initializing. RAM available for dynamic allocation:
I (242) heap_init: At 3FC97078 len 00052698 (329 KiB): RAM
I (248) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (254) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (260) heap_init: At 600FE100 len 00001EE8 (7 KiB): RTCRAM
I (268) spi_flash: detected chip: gd
I (271) spi_flash: flash io: dio
W (275) spi_flash: Detected size(8192k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (288) sleep: Configure to isolate all GPIO pins in sleep state
I (295) sleep: Enable automatic switching of GPIO sleep configuration
I (302) main_task: Started on CPU0
I (322) main_task: Calling app_main()
I (322) example: Create RMT TX channel
I (322) gpio: GPIO[38] InputEn: 0 OutputEn: 1 OpenDrain: 0 Pullup: 1 Pulldown: 0 Intr:0
I (332) example: Create simple callback-based encoder
I (332) example: Enable RMT TX channel
I (342) example: Start LED rainbow chase
```

## Working with the project configuration menu

The **project configuration menu** is a GUI tool running in the terminal that allows you to configure your ESP-IDF project. It's based on Kconfig and offers various low-level **configuration options**, including access to bootloader, serial flash, and security features.

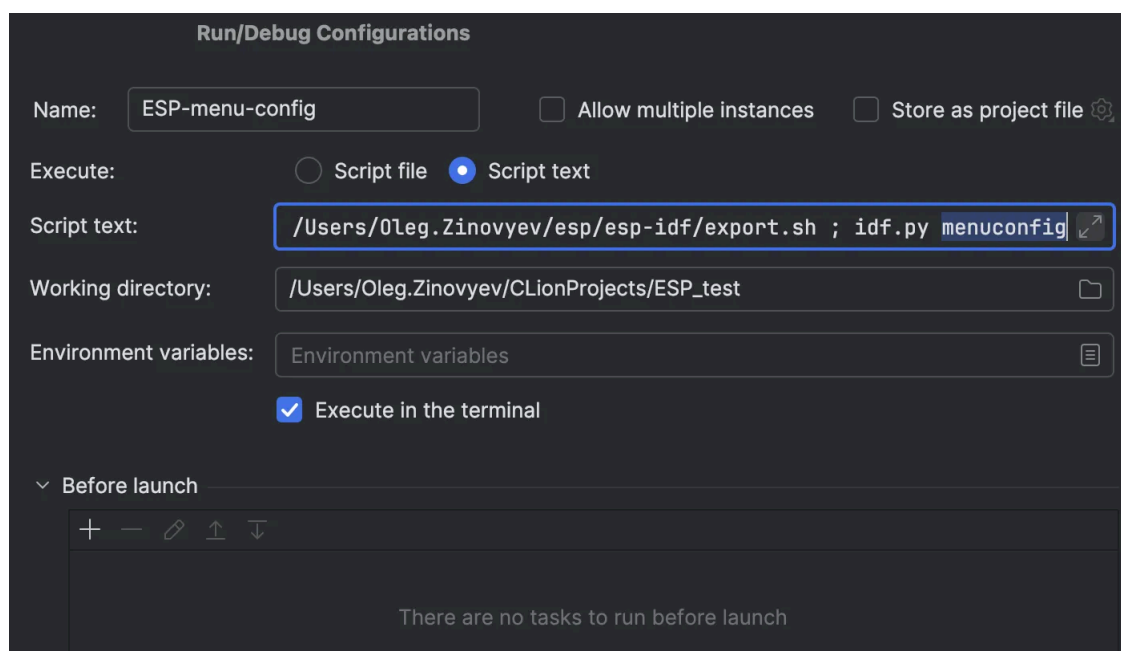
The project configuration menu runs through the `idf.py menuconfig` command, so you need to configure a run configuration accordingly.

1. Open the configuration you've created to run the serial monitor.
2. Click **Copy Configuration**.



3. Rename the copied configuration to reflect the new functionality, for example, **ESP-menu-config**.

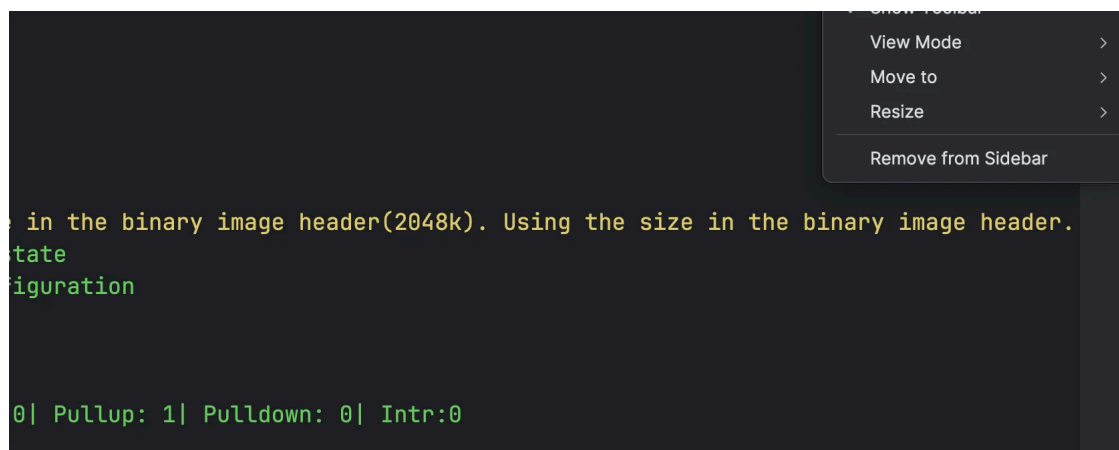
4. In the script text, replace **flash monitor** with **menuconfig**.



5. Click **OK**.

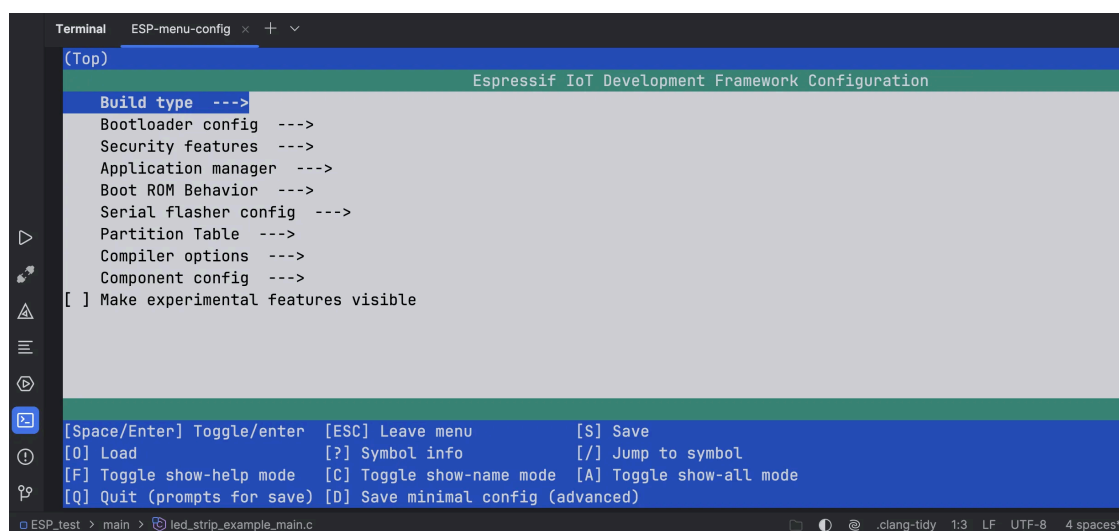
6. Make sure the IDE's new terminal is disabled (unchecked), as the project configuration menu may not work properly with it.



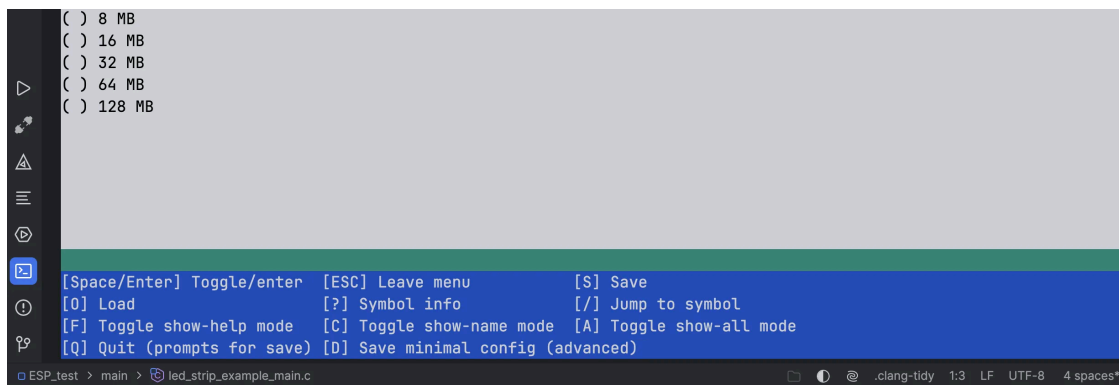


7. Click the green **Run** icon on the main toolbar.

The project configuration menu will open in the IDE's terminal.



You can use the keyboard to navigate the menu and change the default parameters of your project, for example, the flash size.



```

( ) 8 MB
( ) 16 MB
( ) 32 MB
( ) 64 MB
( ) 128 MB

[Space/Enter] Toggle/enter  [ESC] Leave menu      [$] Save
[O] Load              [?] Symbol info    [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

ESP_test > main > led_strip_example_main.c
 clang-tidy 1:3 LF UTF-8 4 spaces*

```

## Using the `idf.py` command in the terminal

You can also use the `idf.py` command with various options in the terminal to manage your project and examine its configuration. For example, here is the output of the `idf.py size` command showing the firmware size:

Terminal ESP-menu-config x + v

Memory Type Usage Summary

| Memory Type/Section | Used [bytes] | Used [%] | Remain [bytes] | Total [bytes] |
|---------------------|--------------|----------|----------------|---------------|
| Flash Code          | 117338       | 1.4      | 8271238        | 8388576       |
| .text               | 117338       | 1.4      |                |               |
| DIRAM               | 61560        | 18.01    | 280200         | 341760        |
| .text               | 47475        | 13.89    |                |               |
| .data               | 11624        | 3.4      |                |               |
| .bss                | 2320         | 0.68     |                |               |
| Flash Data          | 49048        | 0.15     | 33505352       | 33554400      |
| .rodata             | 48792        | 0.15     |                |               |
| .appdesc            | 256          | 0.0      |                |               |
| IRAM                | 16383        | 99.99    | 1              | 16384         |
| .text               | 15356        | 93.73    |                |               |
| .vectors            | 1027         | 6.27     |                |               |
| RTC FAST            | 280          | 3.42     | 7912           | 8192          |
| .rtc_reserved       | 24           | 0.29     |                |               |

Total image size: **241868** bytes (.bin may be padded larger)

0leg.Zinoviyev@CY-UNIT-0167 ESP\_test %

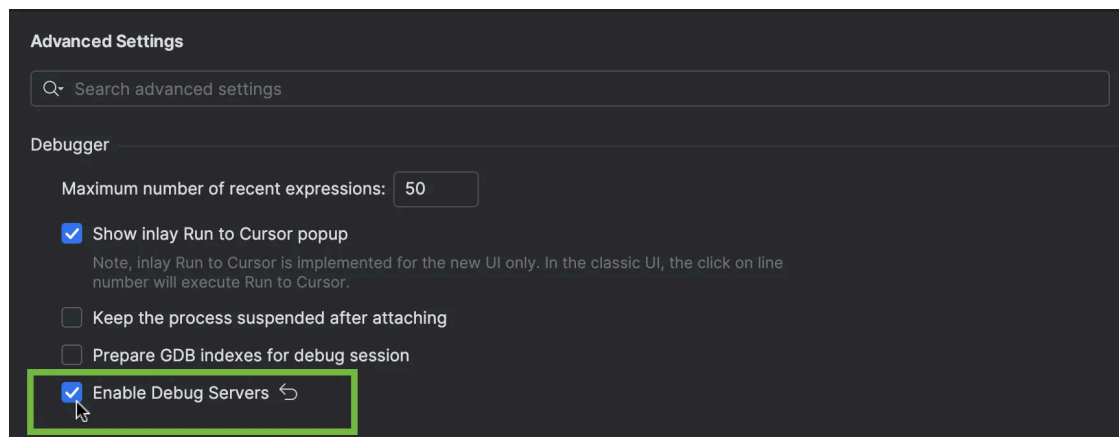
You can configure commands you often use as shell scripts and run them as separate configurations, as you did earlier when accessing the serial monitor port and project configuration menu.

To learn more about the `idf.py` options, read the [official documentation](#).

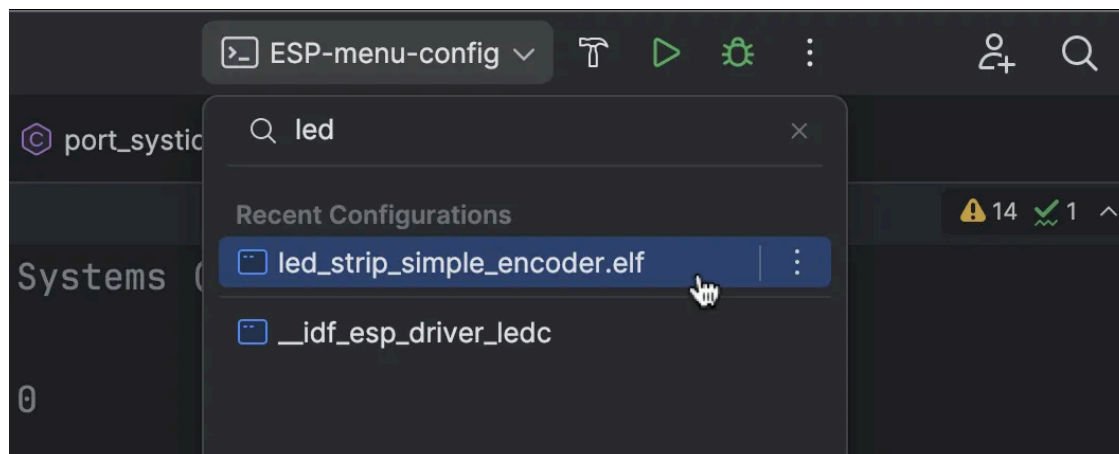
## Debugging a project

with different build targets.

1. Unplug the USB cable from the board's UART connector and plug it into the USB connector.
2. Make sure **Debug Servers** are enabled in **Settings > Advanced Settings > Debugger**.

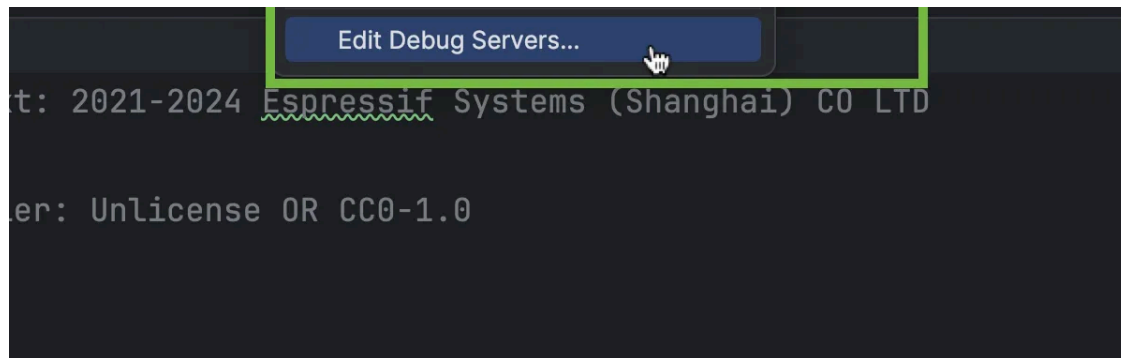


3. Select the **led\_strip\_simple\_encoder.elf** configuration from the main toolbar switcher.

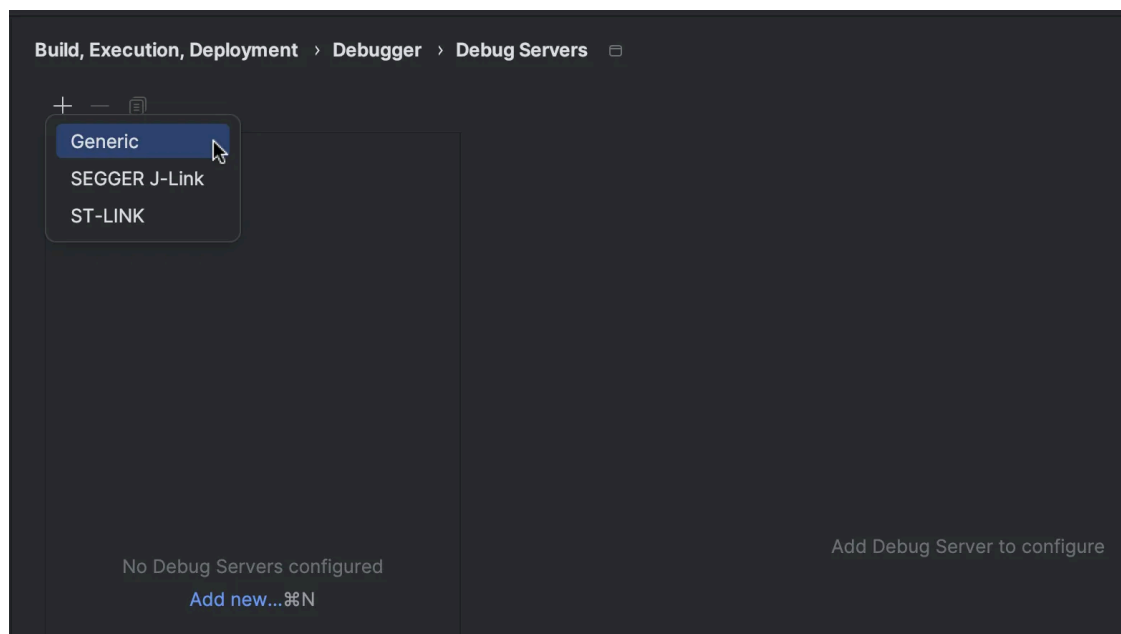


After that, the **Debug Servers** switcher appears in the main toolbar.

4. Select **Edit Debug Servers**.

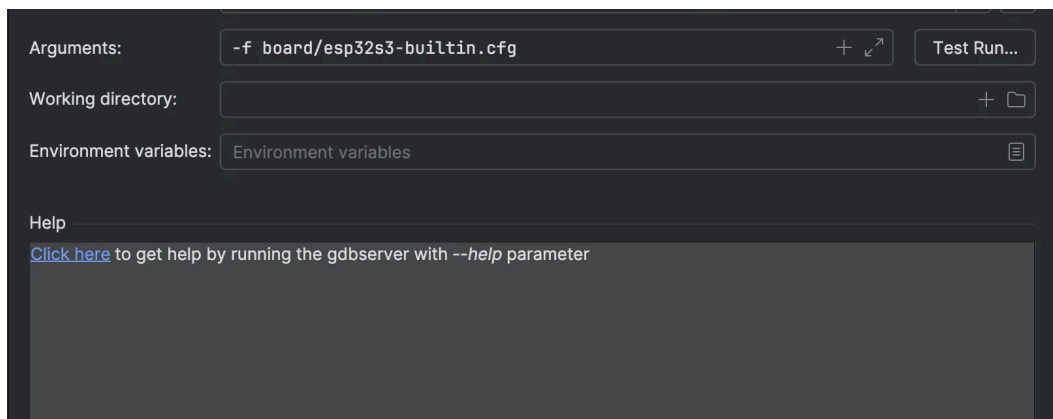


5. Click **+** to add a new debug server.
6. Select the **Generic** template.

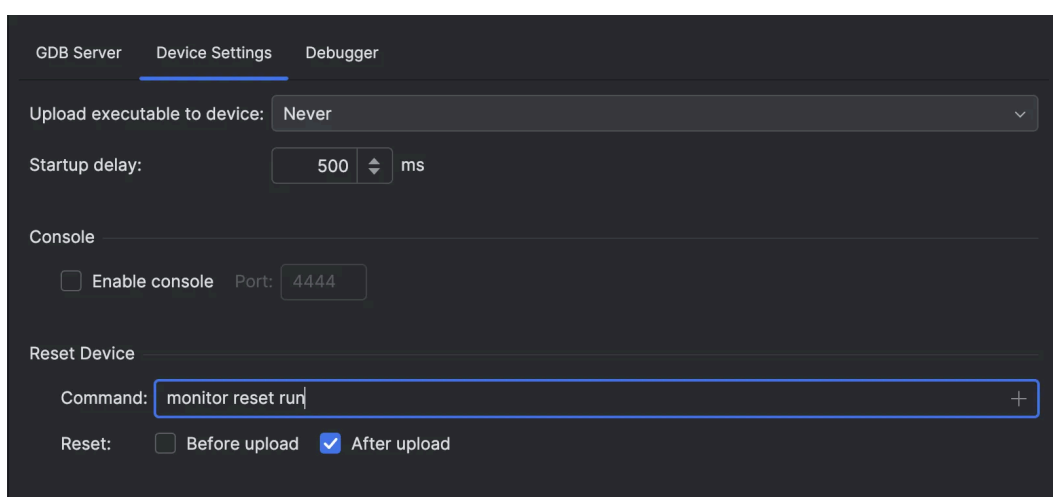


7. Here, you need to specify several parameters, some of which depend on your board. For this tutorial, we'll use the following:

- **GDB Server > Executable :**  
`/Users/Oleg.Zinovyev/.espressif/tools/openocd-esp32/v0.12.0-esp32-20241016/openocd-esp32/bin/openocd`
- **GDB Server > Arguments :** `-f board/esp32s3-builtin.cfg`

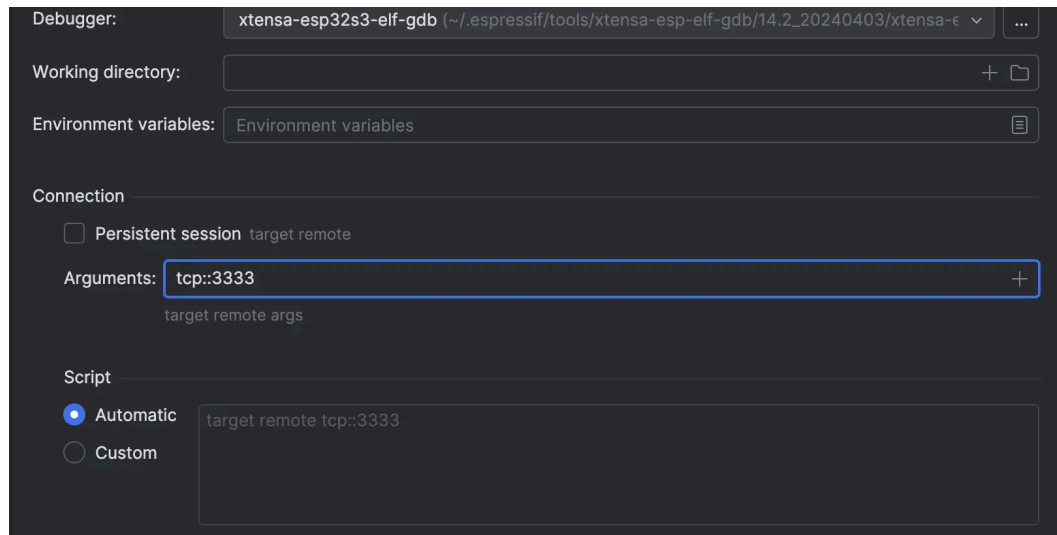


- The **Device Settings** are as follows:



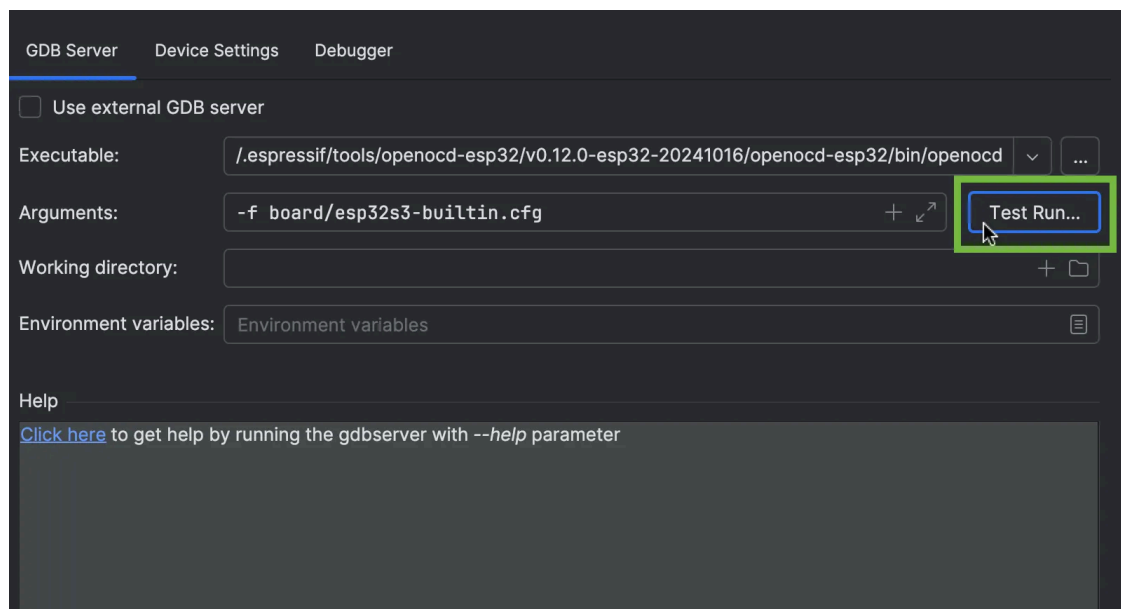
- **Debugger > Custom GDB Executable :**  
/Users/Oleg.Zinovyev/.espressif/tools/xtensa-esp-elf-gdb/14.2\_20240403/xtensa-esp-elf-gdb/bin/xtensa-esp32s3-elf-gdb
- **Debugger > Connection > Arguments :** tcp::3333





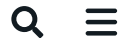
Also, it's best to disable the **Persistent session** option on the **Debugger** tab, as it can be unstable.

8. Leave the rest of the default settings as they are and click **Apply** .
9. You can also run the GDB server in test mode to verify that the settings are correct.



Here is what the **Test Run...** output looks like when the test is successful:





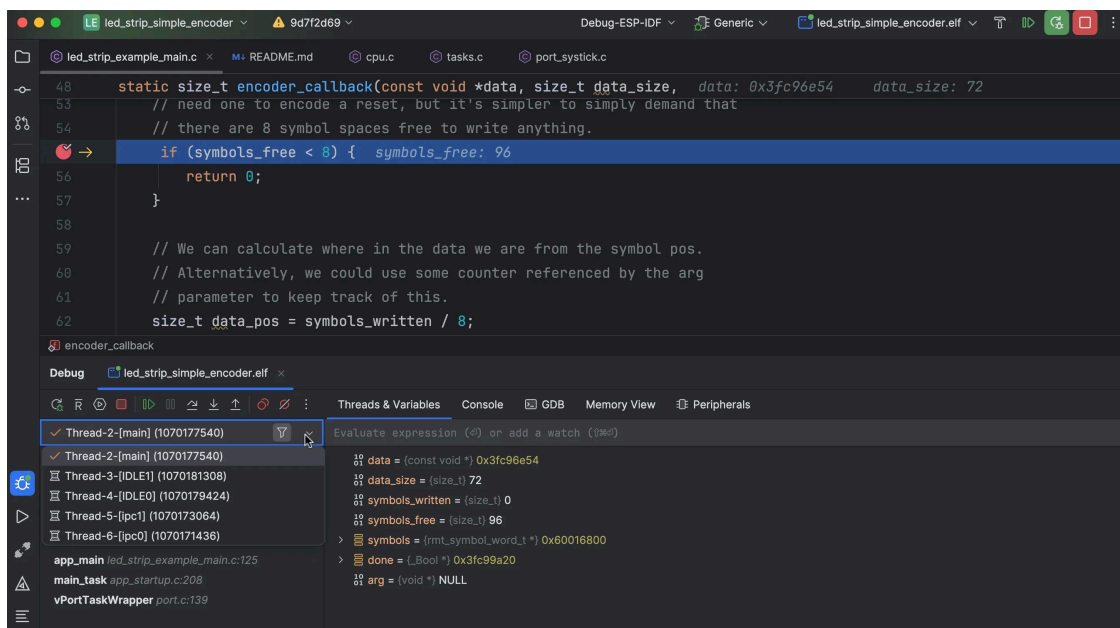
```

For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
Info : only one transport option; autoselecting 'jtag'
Info : esp_usb_jtag: VID set to 0x303a and PID to 0x1001
Info : esp_usb_jtag: capabilities descriptor set to 0x2000
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : esp_usb_jtag: serial (D8:3B:DA:54:5E:48)
Info : esp_usb_jtag: Device found. Base speed 40000KHz, div range 1 to 255
Info : clock speed 40000 kHz
Info : JTAG tap: esp32s3.tap0 tap/device found: 0x120034e5 (mfg: 0x272 (Tensilica), part: 0x2003, ver: 0x1)
Info : JTAG tap: esp32s3.tap1 tap/device found: 0x120034e5 (mfg: 0x272 (Tensilica), part: 0x2003, ver: 0x1)
Info : [esp32s3.cpu0] Examination succeed
Info : [esp32s3.cpu1] Examination succeed
Info : [esp32s3.cpu0] starting gdb server on 3333
Info : Listening on port 3333 for gdb connections
Info : [esp32s3.cpu0] Target halted, PC=0x40378DE6, debug_reason=00000000
Info : [esp32s3.cpu0] Reset cause (15) - (Brown-out core reset)
Info : [esp32s3.cpu1] Target halted, PC=0x40378DE6, debug_reason=00000000
Info : [esp32s3.cpu1] Reset cause (15) - (Brown-out core reset)
  
```

OK

10. Save your changes and close the **Debug Servers** configuration dialog.
11. Set a breakpoint in your source code file.
12. Click the green **Debug** icon on the main toolbar. The debug session will start.

You can then perform the necessary debugging actions and examine the application data.



To learn more about CLion's debugger features, read the **IDE's documentation**.





JTAG settings that you need to be aware of when configuring a debug server for a specific chip.

## Conclusion

At CLion, we strive to make the IDE a universal and convenient tool for developing any embedded system, whether hardware, framework, or toolchain. The same is true for ESP-IDF: We plan to simplify the workflow for these projects and are actively working on it.

We'd appreciate it if you use this tutorial for your ESP-IDF project and give us your feedback. If you have anything to share or if you encounter any problems, please let us know through our [issue tracker](#).



3 reactions



[7 comments](#) · 7 replies – powered by [giscus](#)

Oldest

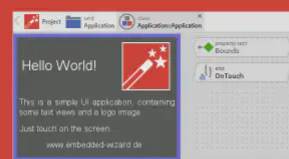
Newest

## Related



**GitHub Copilot**  
with Espressif-IDE

**GitHub Copilot Now in  
Espressif-IDE with**



**GETTING STARTED WITH AN UI APPLICATION  
FOR THE ESP32-S3**

**How to Create an UI  
Application for the**



AI

ESP-IDF

Embedded Wizard

ESP32-S3

ESP32-S2

Using the ESP Flash Download Tool  
to finish the Flash Encryption  
+ Secure Boot  
+ NVS Encryption



## Using the ESP Flash Download Tool to finish the Flash Encryption + Secure Boot + NVS Encryption

23 September 2024 · 22 mins

ESP32-S3

ESP-IDF

Flash Encryption

Secure Boot

NVS Encryption